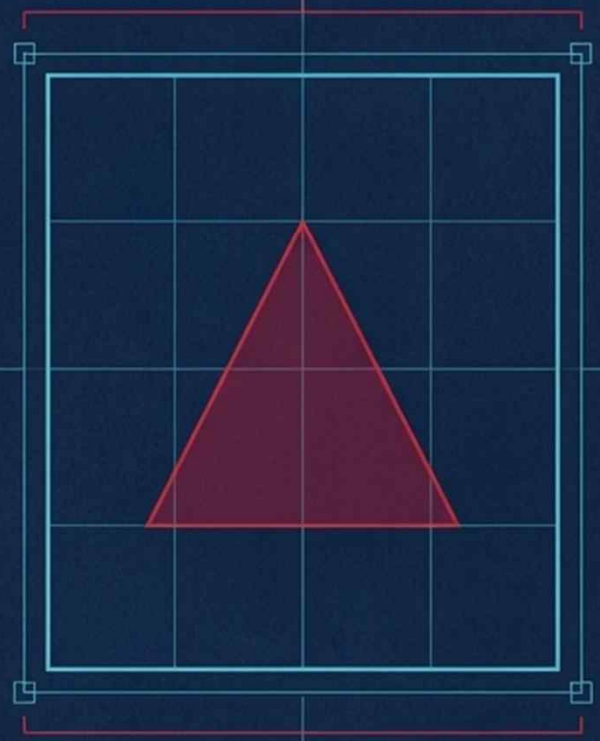
모던 GLSL 렌더링: 파이프라인의 이해와 2D 파티클 실습

MVP 연산과 레거시 코드를 배제한 직접적인 스크린 좌표계(NDC) 활용 및 Quad 파티클 구축



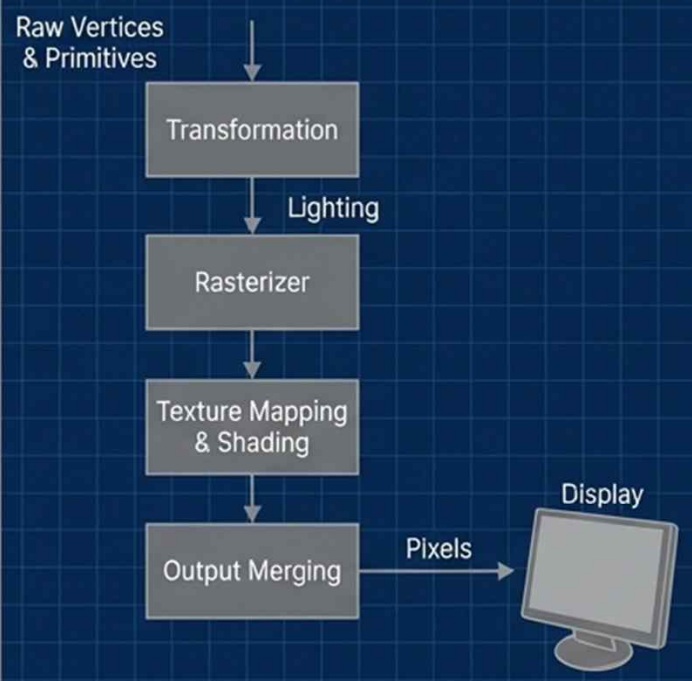
한국공학대학교
TECH UNIVERSITY OF KOREA

그래픽스 파이프라인의 진화

그래픽스 파이프라인은 여러 단계에 걸쳐 진화를 해왔으며, 가장 큰 변화는 '고징(Fixed-function)'에서 '프로그램 가능한(Programmable)' 파이프라인으로의 변화입니다.

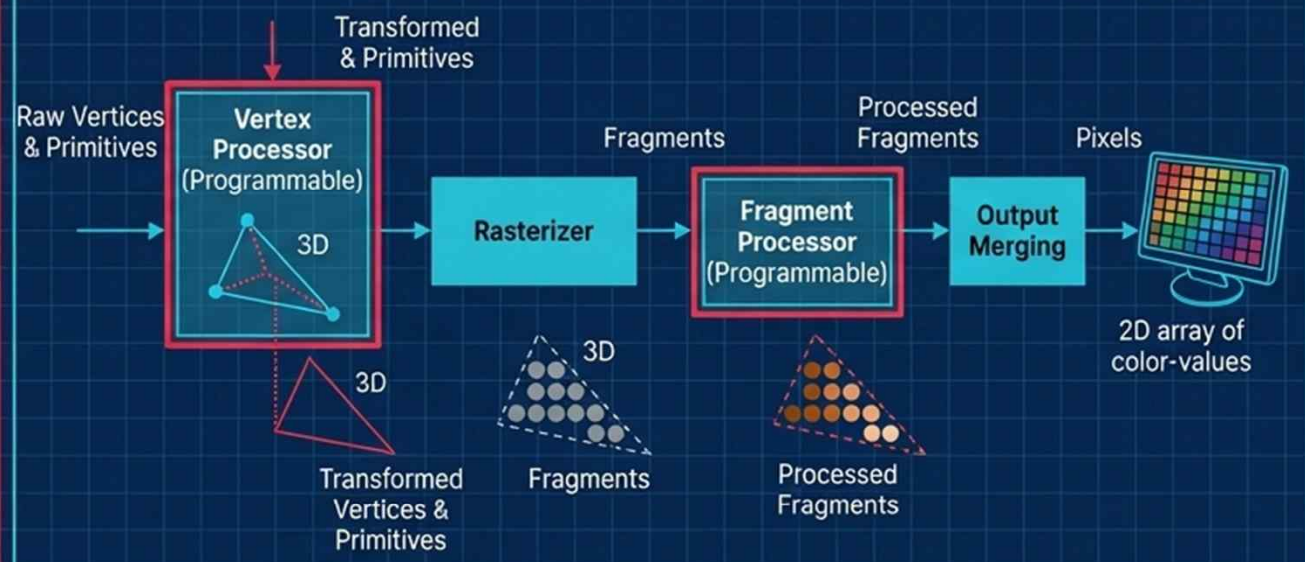
과거: Fixed-Function Pipeline

하드웨어에 고정된 연산 (유연성 부족)



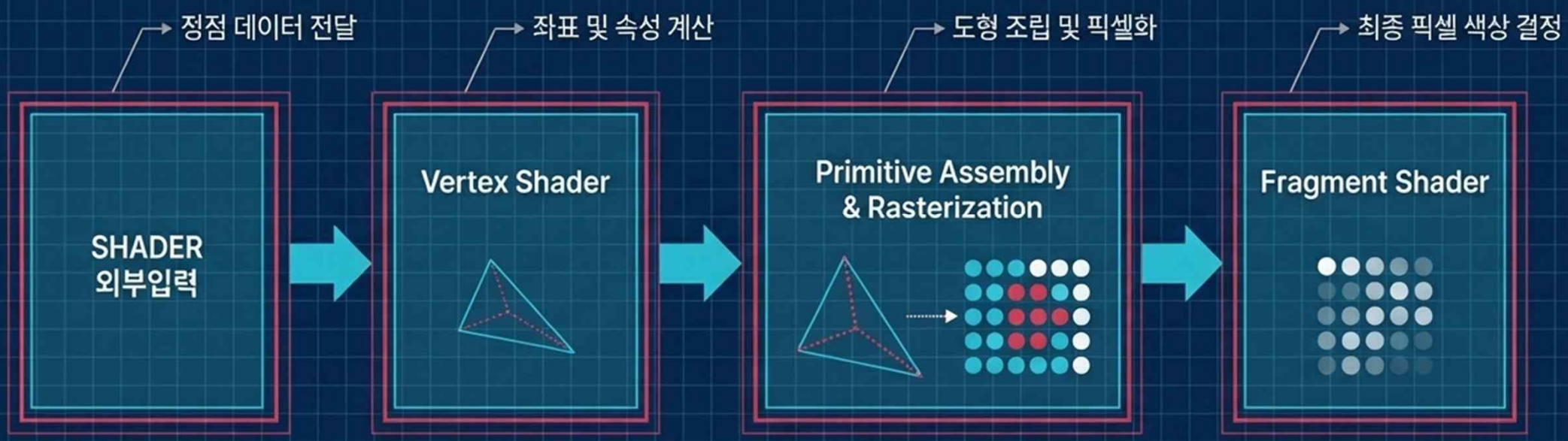
현재: Programmable Pipeline

개발자가 직접 제어하는 세이더(Shader) 프로그래밍



한국공학대학교
TECH UNIVERSITY OF KOREA

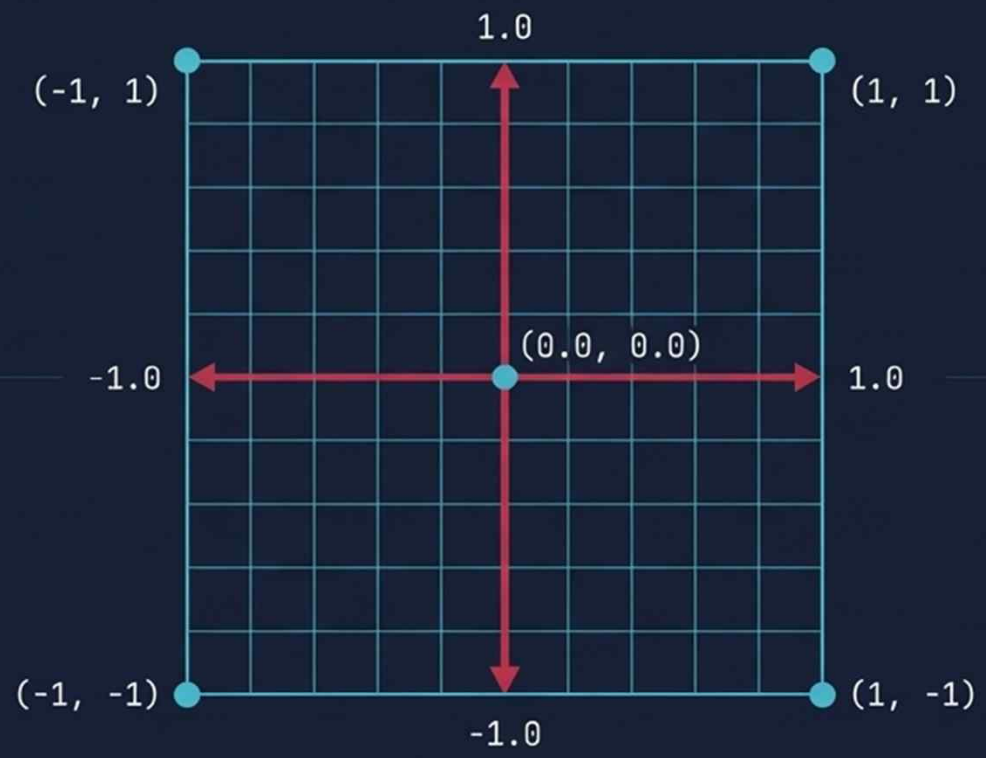
핵심 렌더링 파이프라인 구조



한국공학대학교
TECH UNIVERSITY OF KOREA

우리의 캔버스: NDC 공간 (Normalized Device Coordinates)

복잡한 3D 변환 행렬(MVP) 없이, 스크린에 직접 그리기 위해 우리는 NDC 공간을 절대적인 기준으로 사용합니다.

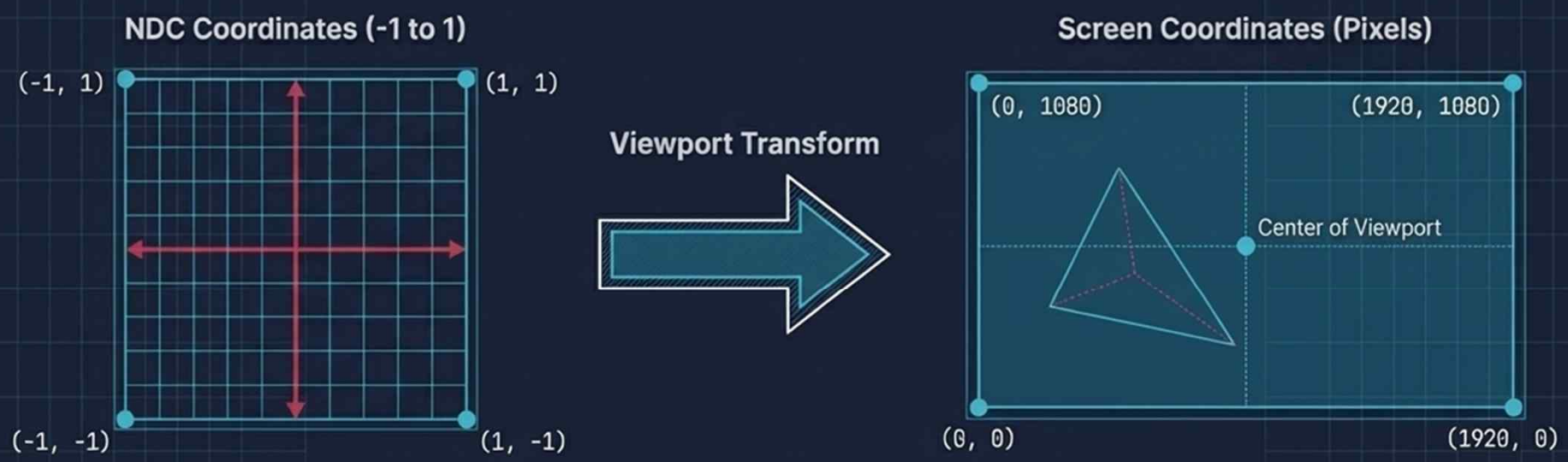


모든 정점 위치는 최종적으로 이 **-1.0 ~ 1.0** 공간 내에 위치해야 화면에 보입니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

Viewport Transformation (화면으로의 투영)

NDC의 정사각형 공간(-1~1)은 **glViewport** 설정을 통해 실제 모니터의 픽셀 해상도(Screen Coordinates)로 자동 변환(스트레칭)됩니다.



Screen Width Viewport Offset X

$$x_window = (width / 2) * (x_{NDC} + 1) + x$$

$$y_window = (height / 2) * (y_{NDC} + 1) + y$$

Screen Height Viewport Offset Y

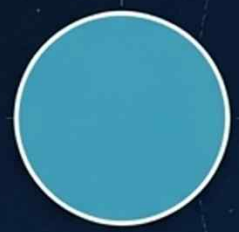
⚠ 주의 (CAUTION)

비율(Aspect Ratio) 보정이 없다면, 창의 비율에 따라 파티클이 길쭉해질 수 있음을 인지해야 합니다.

한국공학대학교 TECH UNIVERSITY OF KOREA

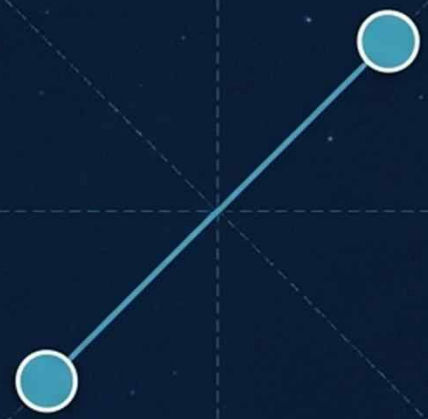
렌더링에 필요한 기본 요소 (Basic Primitives)

OpenGL API는 다양한 기본 도형(Primitive) 조립 방식을 제공합니다.



점 -> Point

```
glDrawArrays(GL_POINTS, 0, 1)
```



선 -> Line

```
glDrawArrays(GL_LINES, 0, 2)
```

삼각형 -> Triangle

```
glDrawArrays(GL_TRIANGLES, 0, 3)
```

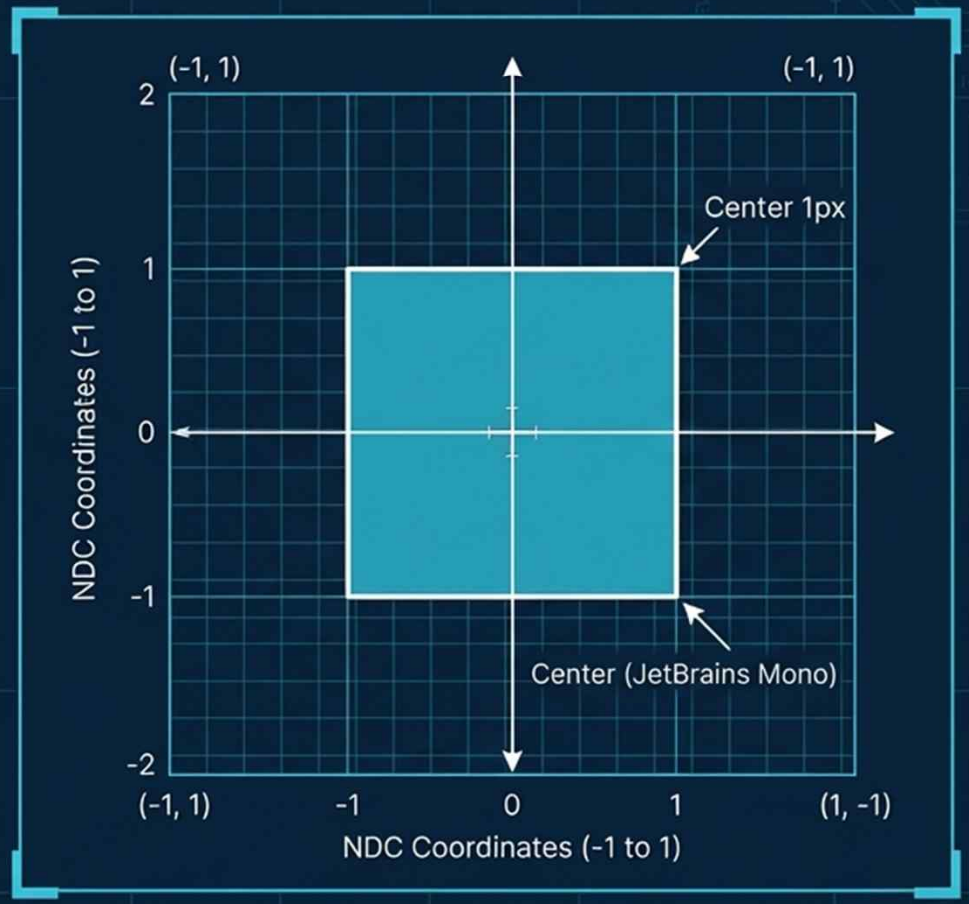
질문: 정육면체나 사각형을 그리기 위해선 몇 개의 정점과 삼각형이 필요할까?

한국공학대학교
TECH UNIVERSITY OF KOREA

실습 목표: 2D 사각형 파티클 렌더링

목표 구현 플랜 (The Plan)

1. MVP 행렬 없이 NDC 좌표 직접 활용
2. 크기와 위치를 Vertex Attribute로 전달
3. GL_POINTS를 대체하는 삼각형(Quad) 기반의 파티클 생성



한국공학대학교
TECH UNIVERSITY OF KOREA

패러다임의 전환: 왜 GL_POINTS를 피해야 하는가?

과거 방식: GL_POINTS + gl_PointSize	모던 방식: GL_TRIANGLES 기반 Quad
• 하드웨어 종속적인 최대 크기 제한 • 텍스처 매핑의 치명적 제약 • 회전(Rotation) 및 형태 변형 불가 • 모던 그래픽스 파이프라인에서의 비권장 방식	• 크기 제한 없음 (화면 전체 덮기 가능) • 완벽한 UV 제어 및 자유로운 텍스처링 • 자유로운 회전, 스케일링 및 변형 • 모든 모던 상용 게임 엔진의 표준 방식

한국공학대학교
TECH UNIVERSITY OF KOREA

모던 파티클의 해답: 2개의 삼각형 (The Quad Anatomy)

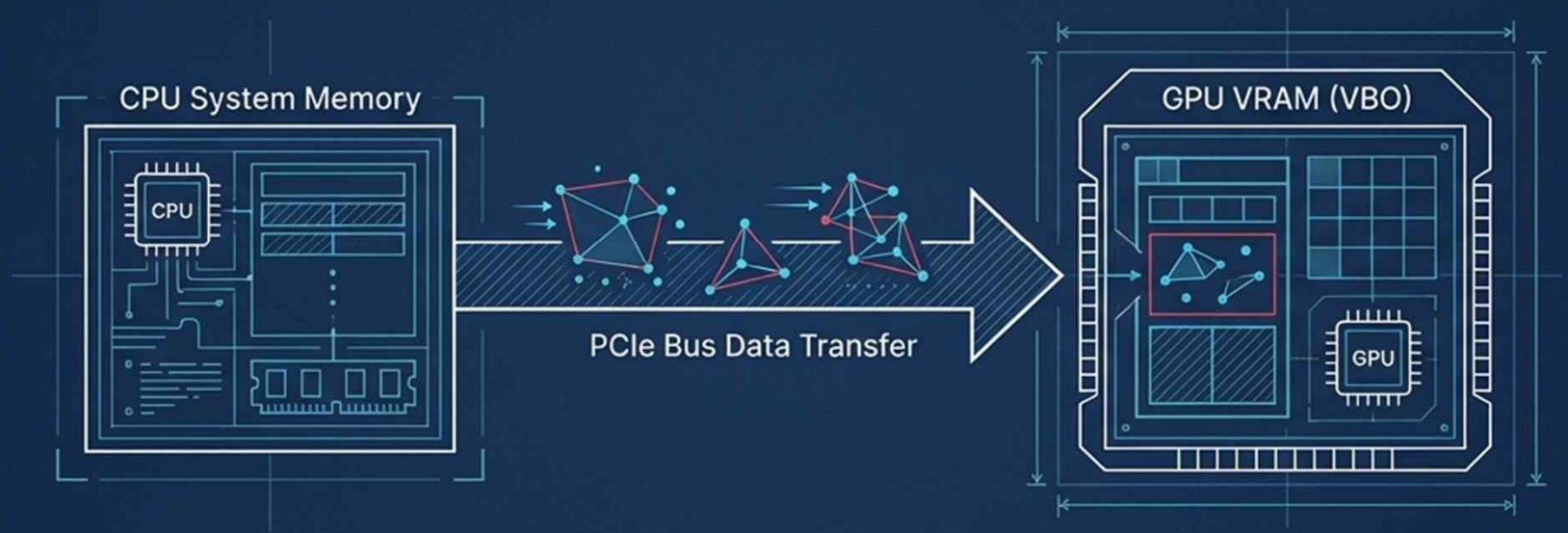
하나의 완벽한 사각형(Quad) 파티클을 만들기 위해
우리는 2개의 직각 삼각형을 결합합니다.

총 6개의 정점(Vertex)을 순서대로 전달하여
2개의 삼각형을 그립니다 (인덱스 버퍼 미사용 시).

한국공학대학교
TECH UNIVERSITY OF KOREA

데이터를 GPU로: VBO (Vertex Buffer Object)

CPU의 시스템 메모리에 있는 정점 데이터를 GPU의 초고속 비디오 메모리로 전송하여 병목 현상을 방지합니다.

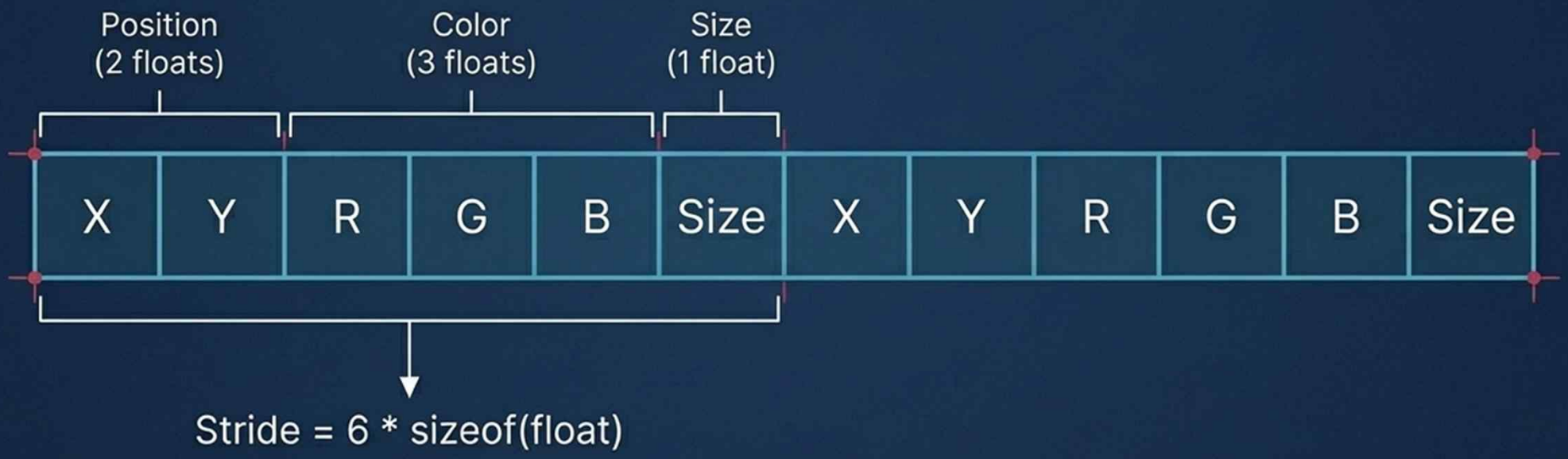



```
glBindBuffer(GL_ARRAY_BUFFER, VBO);
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices), vertices, GL_STATIC_DRAW);
```

한국공학대학교
TECH UNIVERSITY OF KOREA

Quad 파티클을 위한 메모리 설계 (Memory Layout)

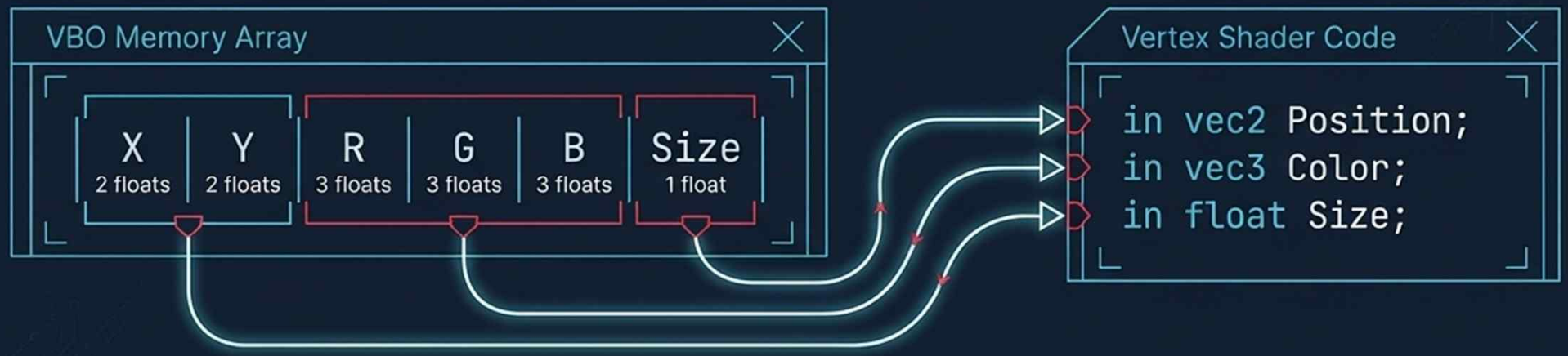
하나의 정점을 표현하기 위해 위치(Position), 색상(Color), 그리고 크기(Size) 데이터를 하나의 배열에 연속적으로 패킹(Packing)합니다.



한국공학대학교
TECH UNIVERSITY OF KOREA

데이터 연결: Vertex Attributes (Wiring the Data)

VBO에 담긴 연속적인 원시 데이터 더미를 셰이더가 이해할 수 있는 변수로 쪼개어 연결해야 합니다.



C++ Code (Attribute Activation Switch)

```
GLint id0 = glGetAttribLocation(program, "Position");
glEnableVertexAttribArray(id0);
```

한국공학대학교
TECH UNIVERSITY OF KOREA

C++ 코드와 하드웨어의 매핑

앞서 설계한 메모리 구조와 셰이더 변수를 실제로 연결하는 C++ API 호출에 대한 상세 분석입니다.

```
glVertexAttribPointer(id0, 2, GL_FLOAT, GL_FALSE, 6 * sizeof(float), (void*)0);
```

- id0 → 셰이더의 위치 변수 ID
- 2 → 요소의 개수 (X, Y 2개)
- GL_FLOAT → 데이터 타입
- GL_FALSE → 정규화(Normalize) 여부
- 6 * sizeof(float) → Stride (다음 정점까지의 바이트 간격)
- (void*)0 → Offset (데이터 시작점 위치)

Takeaway

핵심: API의 각 인자는 앞서 설계한 메모리 구조(Layout)를 그래픽 하드웨어에게 정확히 설명하는 설명서입니다.

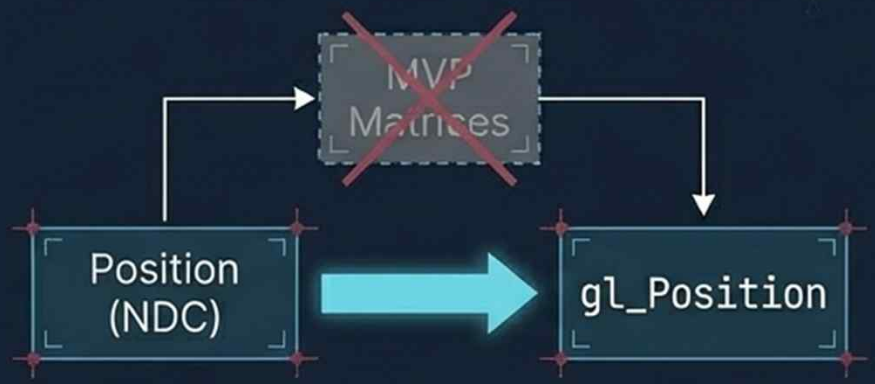
한국공학대학교 TECH UNIVERSITY OF KOREA

Vertex Shader: MVP 없는 직관적 렌더링

Model, View, Projection 행렬을 곱하는 복잡한 연산을 생략하고, CPU에서 넘겨받은 NDC 좌표를 그대로 스크린에 출력합니다.

GLSL

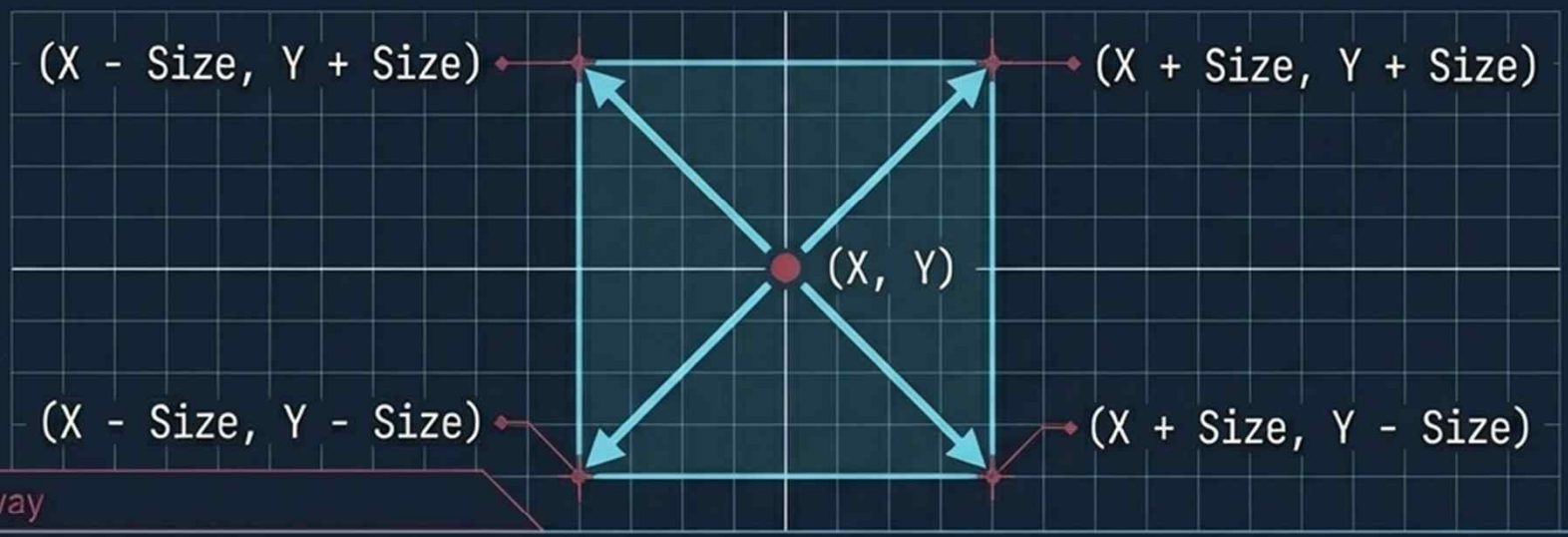
```glsl
#version 330 core
in vec2 Position; // 중심점 (NDC)
in vec3 Color;
in float Size;

out vec3 fragColor;

void main() {
  fragColor = Color;
  // Z와 W는 기본값 할당 (2D 직교 렌더링 효과)
  gl_Position = vec4(Position, 0.0, 1.0);
}
```



한국공학대학교
TECH UNIVERSITY OF KOREA

셰이더 내부에서의 정점 확장 (Calculating Quad Corners)

중앙 위치(Center Position) 하나만 CPU에서 넘기고, 셰이더 내부에서 Size 속성을 더하고 빼서 4개의 모서리 좌표를 계산할 수 있습니다.



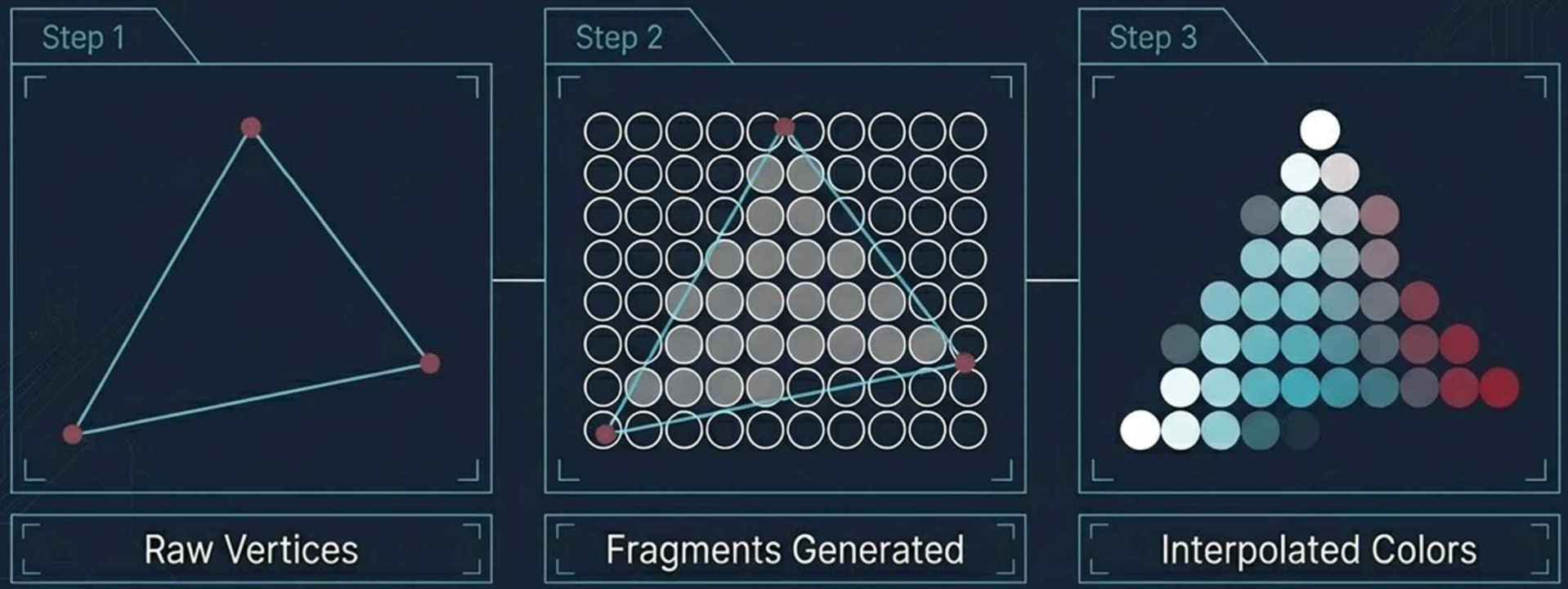
Takeaway

참고: CPU에서 6개의 정점 좌표를 미리 계산해서 넘기는 방식과, 이처럼 셰이더에서 Size로 오프셋을 주는 방식 중 하나를 상황에 맞게 유연하게 적용합니다.

한국공학대학교
TECH UNIVERSITY OF KOREA

Rasterization Phase (래스터화 및 보간)

셰이더 외부 하드웨어에서 일어나는 마법. 계산된 정점들 사이의 빈 공간을 픽셀(Fragment)로 채우고, 색상 데이터를 부드럽게 섞어줍니다(Interpolation).

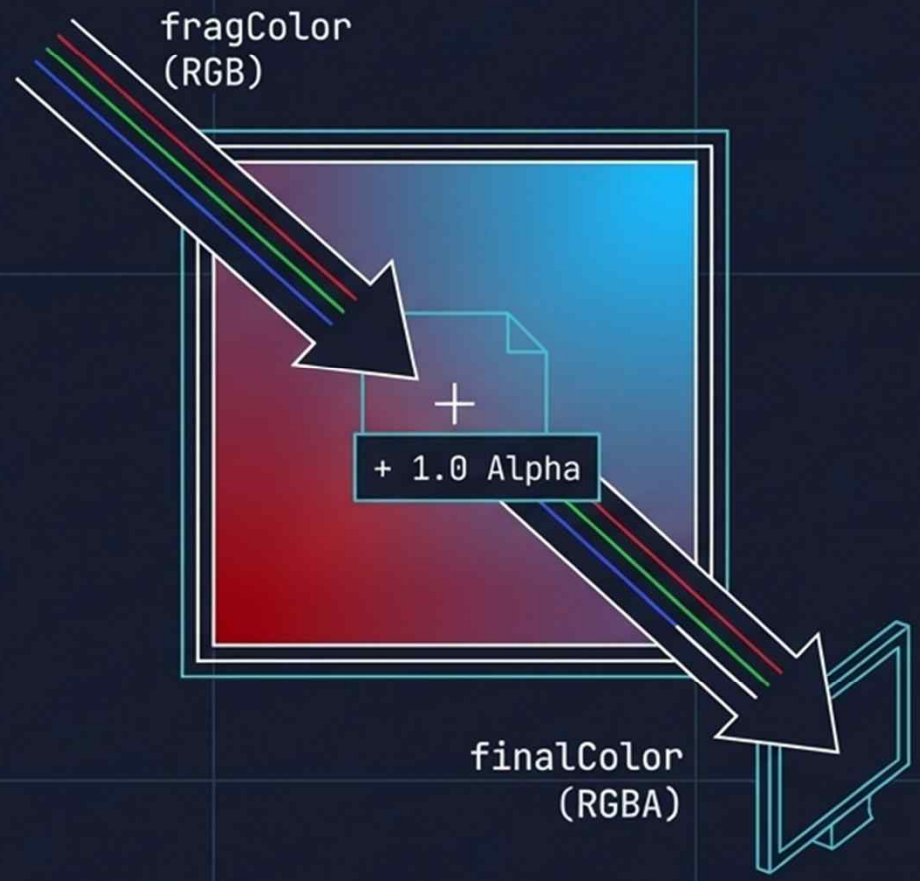


한국공학대학교
TECH UNIVERSITY OF KOREA

Fragment Shader: 파티클 색상 입히기

Rasterizer가 생성한 각각의 픽셀(Fragment)마다 병렬로 실행되며, 보간된(Interpolated) 색상을 화면에 최종 출력합니다.

```
#version 330 core

in vec3 fragColor; // Vertex Shader에서 넘어온 보간된 색상
out vec4 finalColor;

void main() {
    // 알파값 1.0(불투명)을 추가하여 최종 픽셀 색상 결정
    finalColor = vec4(fragColor, 1.0);
}
```



한국공학대학교
TECH UNIVERSITY OF KOREA

렌더링 실행 (Execution: Draw Call)

모든 데이터 준비와 셰이더 연결이 끝나면,
단 한 줄의 명령어로 GPU 파이프라인을 가동시킵니다.

```
glDrawArrays(GL_TRIANGLES, 0, 6);
```

GL_TRIANGLES

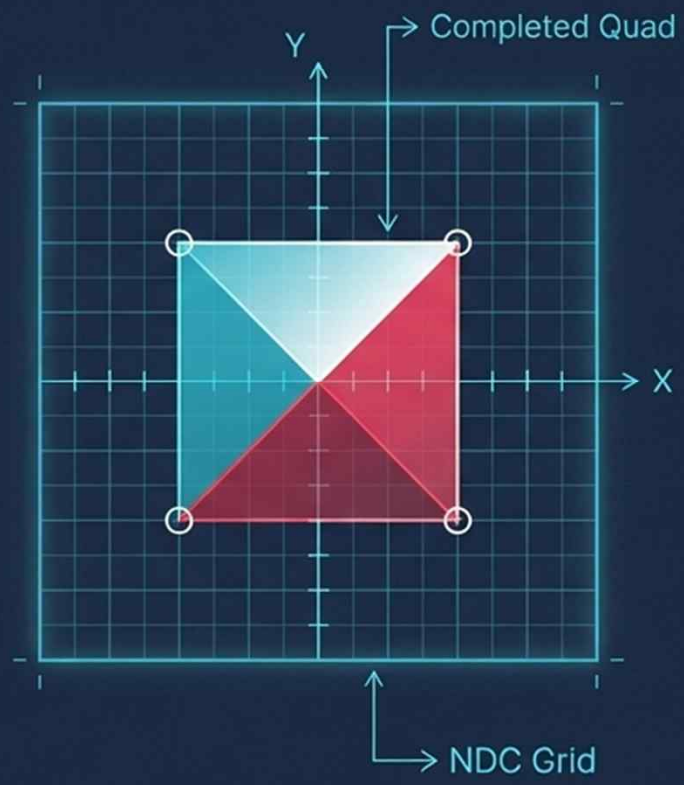
3개씩 묶어서
삼각형을 조립하라.

0

VBO의 0번째
데이터부터 시작하라.

6

총 6개의 정점을 처리하라
(삼각형 2개 = 사각형
파티클 1개 완성).



한국공학대학교
TECH UNIVERSITY OF KOREA

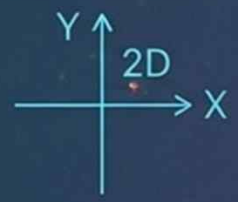
종합: 모던 2D 파티클 렌더링 아키텍처

한국공학대학교
TECH UNIVERSITY OF KOREA

핵심 요약 (Key Takeaways)

1 NDC는 강력한 2D 캔버스다

복잡한 3D 수학(MVP) 없이, -1.0 ~ 1.0 좌표계만으로도
완벽한 2D 화면 제어가 가능합니다.

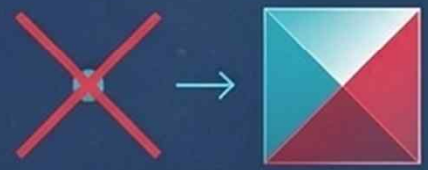


2 모든 것은 데이터의 연결(Attribute)이다

~~gl_PointSize~~ 같은 낡은 내장 변수에 의존하지 않고,
Size와 **Color**를 직접 속성으로 전달하여 완벽한 렌더링 통제권을 집니다.

3 삼각형이 곧 픽셀을 담는 그릇이다

GL_POINTS를 버리고 2개의 삼각형(GL_TRIANGLES)으로
Quad를 구성하는 것이 모던 그래픽스 2D 파티클 렌더링의 절대적 표준입니다.



GL_TRIANGLES = Quad

한국공학대학교
TECH UNIVERSITY OF KOREA